

LAB 5 ADDENDUM · SUPERVISED HOMEWORK**Management Plane Audit, Backup & the Restore Drill**

Who changed the cloud itself, and can you actually get the data back? — reconstructing the audit trail, versioning and a timed recovery on LocalStack

Why This Addendum Exists

Lab 5 taught you to collect and correlate logs from **applications**. This addendum covers the two things a cloud incident responder needs that application logs cannot give you:

- The **management plane** audit trail — not what happened inside a workload, but who reconfigured the cloud itself. Deleting a bucket, disabling a key or attaching an admin policy leaves no application log at all.
- A **tested** recovery. Backup is a claim; restore is a measurement. Until you have timed a restore you do not know your Recovery Time Objective, you only have an aspiration.

Note: This addendum is **supervised homework**. Work through Tasks A1–A4 in your own time, bring your evidence and your measured numbers to the debrief session, and be ready to defend your RTO and RPO figures.

Course & Assessment Mapping

Item	Mapping
Extends	Lab 5 — Monitoring, Logging & Incident Detection (Weeks 9–10)
Course Learning Outcome	CLO2 — Construct secure cloud operations, extended to resilience and recovery (VBE3)
Lecture topics	Week 2 (Security Design & Architecture — management plane) · Week 6 (Monitoring, Auditing & Management)
CSA CCSK v5 domains	Domain 6 (Security Monitoring) · Domain 11 (Incident Response & Resilience)
Assessment	Evidence + measured RTO/RPO submitted with the Lab 5 report; discussed at the debrief

Before You Start

```
docker rm -f localstack 2>/dev/null

# DEBUG=1 makes LocalStack log every AWS API call it serves.
# That request log is the raw material for Task A1.
docker run -d --name localstack -p 4566:4566 \
  -e LOCALSTACK_AUTH_TOKEN=$LOCALSTACK_AUTH_TOKEN \
  -e DEBUG=1 \
  localstack/localstack-pro:latest

# Wait until the edge port is actually serving - the container ID
```

```
# comes back long before the services are ready.
until curl -sf http://localhost:4566/_localstack/health >/dev/null; do sleep 2; done

export EP='--endpoint-url=http://localhost:4566'
aws $EP sts get-caller-identity
```

Task A1 — Reconstruct the Management Plane Audit Trail

Every administrative API call — every create, delete, put-policy, schedule-key-deletion — is a management plane event. In production a trail records them to object storage so the record survives the account being tampered with. Here you will build that trail yourself, seal it, and store it outside the account it audits.

Why not CloudTrail: CloudTrail is not part of the LocalStack licence used in this course — create-trail returns ‘the cloudtrail service is not included within your LocalStack license’. That limitation is itself instructive: the audit trail is a privileged, separately-licensed capability precisely because it is the control an attacker most wants to switch off. In this task you reconstruct the same trail from the platform’s own request log, then apply log file validation by hand. The mechanism you build is what CloudTrail automates; the fields you cannot reconstruct are the ones you will identify at the end of the task.

```
# The audit trail needs its own store. In production this bucket lives
# in a DIFFERENT account - a separate trust boundary from the one it audits.
aws $EP s3api create-bucket --bucket miit-audit-trail
aws $EP s3api put-bucket-versioning --bucket miit-audit-trail \
  --versioning-configuration Status=Enabled

# Mark where the observation window starts, so you capture only
# the administrative activity you are about to generate.
BEFORE=$(docker logs localstack 2>&1 | wc -l)
echo "baseline: $BEFORE lines"
```

Now generate some administrative activity worth auditing — the kind of change that reconfigures the cloud itself rather than the workload running on it.

```
aws $EP s3api create-bucket --bucket miit-throwaway
aws $EP iam create-user --user-name TempContractor
aws $EP iam attach-user-policy --user-name TempContractor \
  --policy-arn arn:aws:iam::aws:policy/AdministratorAccess
aws $EP s3api delete-bucket --bucket miit-throwaway
```

Extract those events from the platform’s own request log. Every API call LocalStack serves is recorded in the form ‘AWS <service>.<Operation> => <status>’, which is the minimum a management plane trail must capture.

```
# Everything the platform recorded since your baseline
docker logs localstack 2>&1 | tail -n +$((BEFORE+1)) \
  | grep -E 'AWS [a-z0-9-]+\.[A-Za-z]+ => ' > mgmt-trail.log
wc -l mgmt-trail.log

# An audit trail is high volume and mostly uninteresting.
# Detection means filtering it down to the calls that change trust.
```

```
grep -E
'\.(CreateUser|AttachUserPolicy|DeleteUser|CreateBucket|DeleteBucket|PutBucketPolicy|ScheduleKeyDeleti
=>' mgmt-trail.log
```

```
# If mgmt-trail.log is empty, the request logger is quiet: restart the
# container with -e LS_LOG=trace in place of -e DEBUG=1 and repeat.
```

Now seal the trail. This is log file validation done by hand — the same tamper-evidence idea you built as a hash chain in Lab 5, Task 4. The digest must live in the audit store, not beside the log, or an attacker who edits one simply edits the other.

```
# Seal it (macOS without coreutils: use shasum -a 256 instead)
sha256sum mgmt-trail.log > mgmt-trail.sha256
cat mgmt-trail.sha256

# Ship both to the audit store - the separate trust boundary
aws $EP s3 cp mgmt-trail.log s3://miit-audit-trail/
aws $EP s3 cp mgmt-trail.sha256 s3://miit-audit-trail/

# The attacker edits the trail to hide the privilege grant
grep -v 'AttachUserPolicy' mgmt-trail.log > t.log && mv t.log mgmt-trail.log

# Verify against the digest fetched BACK from the audit store
aws $EP s3 cp s3://miit-audit-trail/mgmt-trail.sha256 ./check.sha256
sha256sum -c check.sha256 # macOS: shasum -a 256 -c check.sha256
# Expect: mgmt-trail.log: FAILED -> the alteration is detected
```

Your reconstruction proves that a call happened; it does not say who made it, from where, or with what arguments. Real CloudTrail writes one JSON record per API call. The record below is a specimen for reading, not something your lab will produce — it is what AWS would have logged for the third command you ran above, `aws iam attach-user-policy`, which granted `AdministratorAccess` to the user `TempContractor`. The account number and IP address in it are documentation-reserved examples.

```
{
  "eventVersion": "1.09",
  "userIdentity": {
    "type": "IAMUser",
    "arn": "arn:aws:iam::123456789012:user/j.tan",
    "accountId": "123456789012",
    "userName": "j.tan",
    "sessionContext": { "mfaAuthenticated": "false" }
  },
  "eventTime": "2026-03-01T09:01:40Z",
  "eventSource": "iam.amazonaws.com",
  "eventName": "AttachUserPolicy",
  "awsRegion": "us-east-1",
  "sourceIPAddress": "203.0.113.9",
  "userAgent": "aws-cli/2.15.0 Python/3.11 Linux/6.5",
  "requestParameters": {
    "userName": "TempContractor",
    "policyArn": "arn:aws:iam::aws:policy/AdministratorAccess"
  },
}
```

```

"errorCode": null,
"readOnly": false,
"managementEvent": true,
"eventID": "1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d"
}

```

Verify or explain: Put your mgmt-trail.log line for AttachUserPolicy beside the record above and answer in writing. (a) Name four fields the real record carries that your reconstruction does not, and state what each one lets an investigator establish that they otherwise could not. (b) The sourceIPAddress in the record is the same address that brute-forced the login in Lab 5 — explain what that single field turns two separate investigations into. (c) Your trail was written by the same platform that served the calls; name one way an attacker with administrator rights could defeat it, and what a real deployment does to prevent that.

Security tip: Notice which of those four calls an application log would have captured: none of them. attach-user-policy granting administrator rights produces no application traffic, no failed login and no anomalous request — the workload never sees it. The management plane is a separate telemetry source and needs separate collection, separate storage and separate alerting. That is the point of this task.

Task A2 — Backup, and Why Versioning Is Not One

Set up a primary store and a genuinely separate backup destination.

```

aws $EP s3api create-bucket --bucket miit-primary
aws $EP s3api create-bucket --bucket miit-dr-backup

aws $EP s3api put-bucket-versioning --bucket miit-primary \
  --versioning-configuration Status=Enabled
aws $EP s3api put-bucket-versioning --bucket miit-dr-backup \
  --versioning-configuration Status=Enabled

# A dataset large enough that the restore takes measurable time
for i in $(seq 1 200); do
  echo "patient record $i - $(date)" > /tmp/rec$i.txt
done
aws $EP s3 sync /tmp/ s3://miit-primary/records/ --exclude '*' --include 'rec*.txt'

aws $EP s3 ls s3://miit-primary/records/ | wc -l

# Take the backup
aws $EP s3 sync s3://miit-primary s3://miit-dr-backup

aws $EP s3 ls s3://miit-dr-backup/records/ | wc -l

```

Caution: In Lab 6 you enabled versioning and saw that a deleted object was still recoverable. It is tempting to conclude that versioning is a backup. It is not. Versioning protects an object **inside** a bucket; it does not survive the bucket being deleted, the account being compromised, or an attacker with s3:DeleteObjectVersion. A backup is a copy in a different trust boundary. Write one sentence in your report naming a failure that versioning survives and one it does not.

Task A3 — The Restore Drill (Timed)

Simulate a destructive incident, then recover — and time it. The number you produce here is your measured RTO for this dataset at this size.

```
# The incident
aws $EP s3 rm s3://miit-primary/records/ --recursive
aws $EP s3 ls s3://miit-primary/records/ | wc -l      # expect 0

# The recovery, timed
START=$(date +%s)
aws $EP s3 sync s3://miit-dr-backup s3://miit-primary
END=$(date +%s)

echo "Objects restored: $(aws $EP s3 ls s3://miit-primary/records/ | wc -l)"
echo "MEASURED RTO (seconds): $((END - START))"
```

Record three numbers in your report:

Measure	How you obtain it	Your value
Measured RTO	The elapsed seconds printed above, for 200 objects	
Extrapolated RTO	Scale your measurement to 1,000,000 objects and state your assumptions	
RPO	The time between your last s3 sync and the incident — data written in that window is lost	

Security tip: The extrapolation matters more than the measurement. A restore that takes four seconds for 200 objects is not reassuring if the production dataset is five thousand times larger and your board has been promised a four-hour RTO. State the assumption you are least confident about — usually that throughput scales linearly, which it does not.

Task A4 — Compare the Two Recovery Paths

You now have two ways to recover the same data. They are not equivalent. Run both and compare.

```
# Path 1 - versioning: the delete only wrote delete markers, so the
# objects are still in the primary bucket underneath them
aws $EP s3api list-object-versions --bucket miit-primary \
  --prefix records/ --query 'length(DeleteMarkers)'

aws $EP s3api list-object-versions --bucket miit-primary \
  --prefix records/ --query 'length(Versions)'

# Path 2 - restore from the separate backup bucket (what you did in A3)
```

Complete this comparison in your report. This table is the deliverable the debrief session is built around.

	Versioning (in-place)	Separate backup bucket
Recovery speed		
Survives bucket deletion?		

Survives a compromised admin credential?		
Survives cryptographic erasure of the KMS key?		
Cost profile		

Deliverables

1. Evidence (label each clearly)

- The filtered management plane trail, its sealed digest, and the failed verification after tampering (A1).
- Object counts in the primary and backup buckets after the sync (A2).
- The incident, the timed restore, and the MEASURED RTO line (A3).
- The delete-marker and version counts from both recovery paths (A4).
- The completed RTO/RPO table and the completed recovery-path comparison.

2. Short-Answer Questions

1. Name three administrative actions that would appear in a management plane trail but produce no application log at all. For each, state what an attacker gains by performing it.
2. CloudTrail log file validation, the digest you sealed in Task A1, and the hash chain you built in Lab 5, Task 4 all solve the same problem by the same mechanism. Explain the mechanism, and state why the digest must be written to a different trust boundary from the account it audits.
3. Distinguish RTO from RPO using your own measured figures. Which of the two is improved by taking backups more frequently, and which by restoring faster?
4. Your measured RTO was a few seconds. Explain why you should not report that number to a board, and what you would report instead.
5. Using your A4 table, state one incident that versioning survives and the separate backup does not, and one that the backup survives and versioning does not.

For the Debrief Session (30 minutes)

Come prepared to answer these. They are the discussion, not a written submission.

- What is your extrapolated RTO, and what assumption is it most sensitive to?
- Your organisation promises clients a four-hour RTO. Based on your numbers, is that promise defensible? What would you need to change to make it so?
- Who should be alerted when `attach-user-policy` grants administrator rights, and how quickly?
- If an attacker obtains an administrator credential, which of your controls still holds — and for how long?

Cleanup

```
aws $EP s3 rb s3://miit-dr-backup --force
aws $EP s3 rb s3://miit-audit-trail --force
aws $EP iam detach-user-policy --user-name TempContractor \
```

```
--policy-arn arn:aws:iam::aws:policy/AdministratorAccess
aws $EP iam delete-user --user-name TempContractor
docker rm -f localstack
rm -f /tmp/rec*.txt mgmt-trail.log mgmt-trail.sha256 check.sha256
```

References

- Course lectures — Week 2 (Security Design & Architecture); Week 6 (Monitoring, Auditing & Management).
- AWS CloudTrail log file integrity validation — docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html
- CSA Security Guidance v5 — Domain 6 (Security Monitoring) and Domain 11 (Incident Response & Resilience).
- Lab 5 — Monitoring, Logging & Incident Detection; Lab 6 — Object Storage Security (versioning and delete markers).